Routing: params, query, and Routers

9 min read · Updated 24 Jul 2026 · Topic 35 of 70

Bookmark · Mark done

Why it matters

Routing is how a server decides *which piece of your code answers a given URL*. Every REST API is, at bottom, a routing table: `GET /users` lists users, `POST /users` creates one, `GET /users/:id` fetches one, `DELETE /users/:id` removes it. Get routing right and your API reads like a map of your domain; get it wrong and you end up with a tangle of conflicting paths, parameters that read the wrong value, and a `server.js` that is two thousand lines long. Interviewers lean on routing because it reveals whether you can structure an application, not just make one endpoint respond.

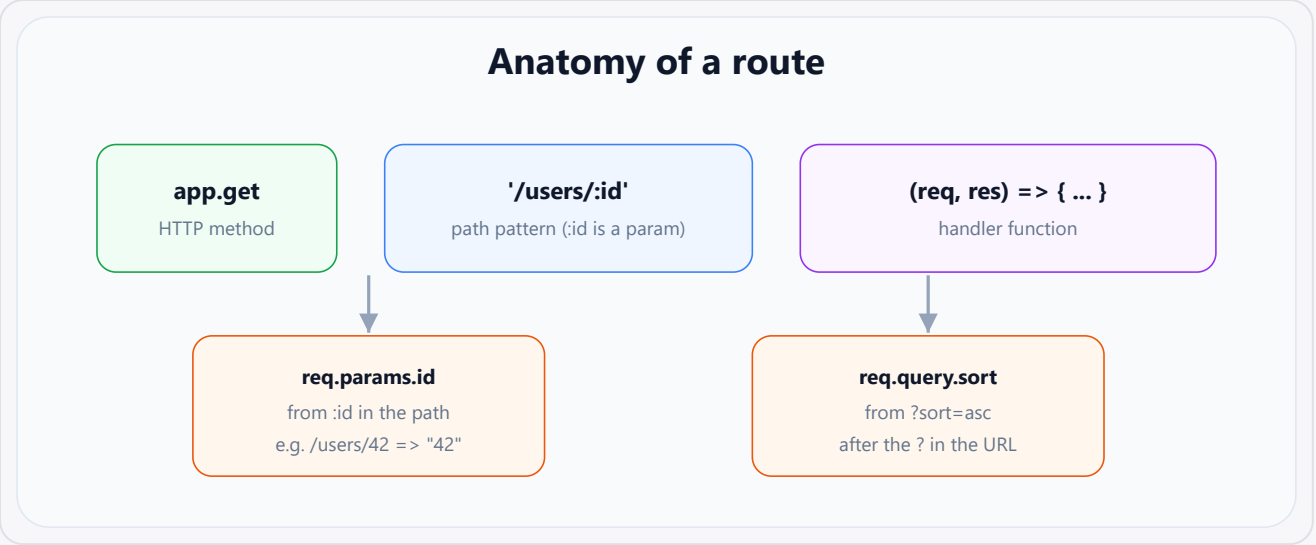
This topic covers the four things you will use constantly: defining routes for HTTP methods, reading dynamic *route parameters* (`:id`), reading the *query string* (`?sort=asc`), and — the part that separates toy apps from real ones — splitting routes into modular **Routers** so a growing codebase stays navigable. Mastering these turns "I can write an endpoint" into "I can architect an API."

The concept, simply

A route in Express is a triple: an **HTTP method**, a **path pattern**, and one or more **handler functions**. You register one with `app.get(path, handler)`, `app.post( ... )`, `app.put`, `app.patch`, `app.delete`, and so on — one method per verb, plus `app.all()` for every method. The path pattern can be a literal string (`'/health'`) or contain named parameters marked with a colon (`'/users/:id'`). When a request matches, Express calls the handler.

Two sources of variable data ride along with a request. **Route parameters** are parts of the path itself: in `/users/:id`, the `:id` segment captures whatever is in that position, so `/users/42` gives you `req.params.id === '42'`. The **query string** is the part after

paginates a collection.



A route is method + path + handler; the :id segment lands in req.params, the ?sort part lands in req.query.

As an app grows you do not want every route in one file. A **Router** is a mini, mountable Express app: you create it with `express.Router()`, attach routes to it, and mount it under a base path with `app.use('/api/users', usersRouter)`. Inside the router, paths are written relative to that base, so `router.get('/:id')` answers `GET /api/users/:id`. This is how real projects keep a hundred endpoints organised.

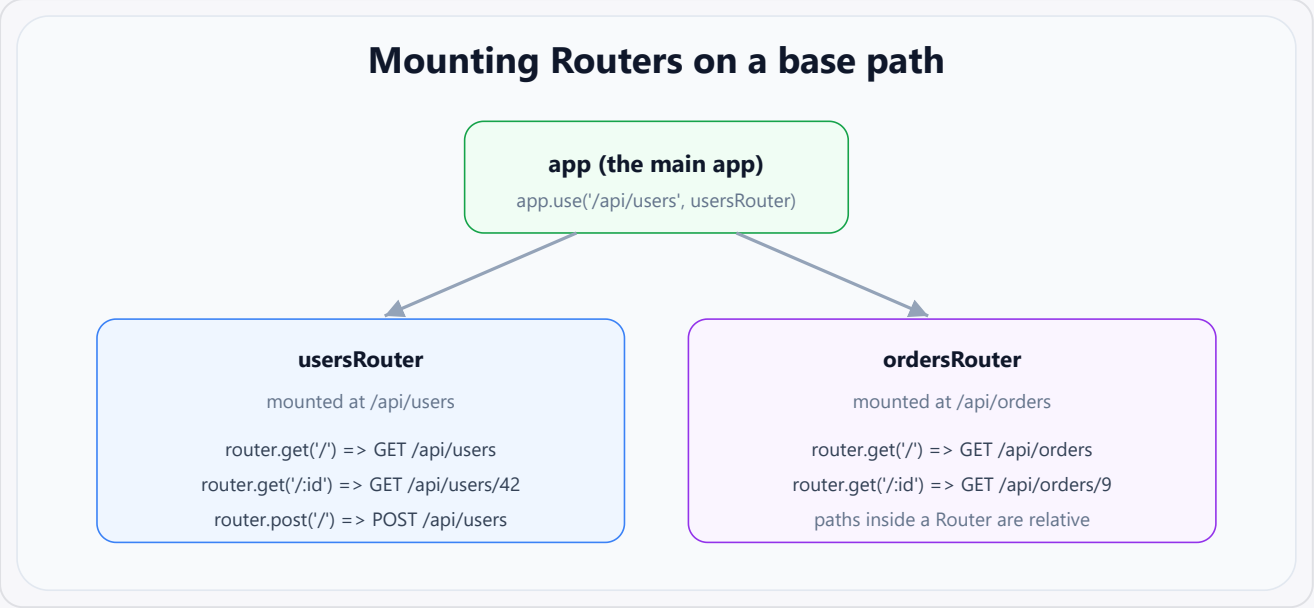
How it works

Under the hood the router is an ordered list, and Express matches routes **top to bottom, first match wins for that step**. When a request arrives, Express compares its method and path against each route in registration order. The *first* route whose handler responds (without calling `next()`) ends processing. This ordering has a concrete consequence: a more specific route must be registered *before* a more general one that would also match, or the general one shadows it. `/users/new` must come before `/users/:id`, otherwise `:id` greedily captures the string `"new"`.

Path patterns are compiled into regular expressions by a library (`path-to-regexp`). A segment like `:id` becomes a capture group; its matched text is stored under that name in `req.params`. Note two important facts: **route params are always strings** (`'42'`, never `42` — you convert with `Number()` yourself), and they are captured per-

segments and unnamed wildcards are written — but named `:param` routing is unchanged.

The query string is parsed separately from the path. Express reads everything after the `?`, decodes it, and builds `req.query`. Repeated keys and bracket syntax can produce arrays and nested objects (`?tag=a&tag=b` → `{ tag: ['a', 'b'] }`), which is powerful but also a source of surprises — a value you expect to be a string can arrive as an array if a client sends the key twice. Everything in `req.query` is also a string (or array/object of strings); there is no type coercion.



A Router is a mini-app mounted at a base path; routes inside it are written relative to that base and Express joins them.

When you mount a router with `app.use('/api/users', usersRouter)`, Express treats the router itself as one middleware entry keyed to that base path. A request to `/api/users/42` matches the mount, Express strips the base (`/api/users`) and passes the remainder (`/42`) into the router, which then does its own top-to-bottom matching against `/:id`. Routers can be nested and can carry their own middleware (`router.use( ... )`), so cross-cutting concerns like "authenticate every admin route" attach cleanly to a subtree instead of the whole app.

Code examples

JAVASCRIPT Copy

```
// GET /users/42?fields=name
app.get('/users/:id', (req, res) => {
  const id = Number(req.params.id);        // '42' (string) → 42
  const fields = req.query.fields;         // 'name'
  res.json({ id, fields });
});

app.post('/users', (req, res) => {
  res.status(201).json({ created: req.body });
});

app.delete('/users/:id', (req, res) => {
  res.status(204).end();
});
```

2. Ordering matters — specific before generic.

JAVASCRIPT Copy

```
// CORRECT: the literal route is registered first
app.get('/users/me', (req, res) => res.json({ self: true }));
app.get('/users/:id', (req, res) => res.json({ id: req.params.id }));

// If reversed, GET /users/me would match /users/:id
// and req.params.id would be the string "me".
```

3. A Router in its own module — the real-world pattern.

JAVASCRIPT Copy

```
// routes/users.js
const express = require('express');
const router = express.Router();

// middleware for THIS router only
router.use((req, res, next) => { req.scope = 'users'; next(); });
```

```
module.exports = router;
```

JAVASCRIPT Copy

```
// app.js
const express = require('express');
const app = express();
const usersRouter = require('./routes/users');

app.use(express.json());
app.use('/api/users', usersRouter);   // mount the router at a base path
app.listen(3000);
```

4. Chaining methods on one path with `app.route()` to avoid repeating the path string.

JAVASCRIPT Copy

```
app.route('/books/:id')
  .get((req, res) => res.json({ read: req.params.id }))
  .put((req, res) => res.json({ updated: req.params.id }))
  .delete((req, res) => res.status(204).end());
```

Common mistakes & gotchas

- **Registering** `:id` **before a literal sibling.** `/users/:id` placed above `/users/me` swallows `/users/me` — `req.params.id` becomes `"me"`. Specific routes first.
- **Treating params as numbers.** `req.params.id` is always a string. `req.params.id === 42` is `false`; convert with `Number()` and validate.
- **Assuming a query value is a string.** A duplicated key (`?tag=a&tag=b`) yields an array, and bracket syntax yields objects. Defensive code checks the type before using it.

- **Duplicate route definitions.** Defining the same method+path twice: the first that responds wins, the second is dead code — no error is thrown, so it hides quietly.
- **Porting Express 4 wildcard/regex paths to 5 unchanged.** The matcher changed; unnamed wildcards and optional markers were rewritten. Named `:param` routes are safe, but exotic patterns need review.

Best practices

- **One Router per resource** (`users`, `orders`, `products`) in its own file under a `routes/` folder, each mounted at `/api/<resource>`.
- **Order routes specific → generic** within a file, and keep the mount base path in `app.use` so the router's internal paths stay short and relative.
- **Validate and coerce params and query at the boundary** (a small middleware or a schema library like `zod` / `joi`) so handlers receive clean, typed data.
- **Use `app.route()` or method chaining** when several verbs share one path, to keep the path defined once.
- **Keep handlers thin** — parse the request, call a service/controller function, send the response. Business logic does not belong inside the route callback.
- **Use `express.Router({ mergeParams: true })`** when a nested router needs params from its parent's mount path.

⭐ Interview lens

> **Q: "What is the difference between `req.params` and `req.query`?"**
> A: `req.params` holds named segments from the path pattern (`/users/:id` → `req.params.id`) and identifies a specific resource. `req.query` holds the parsed query string after the `?` and typically filters, sorts, or paginates. Both arrive as strings — Express does no type coercion.

A: Express matches top to bottom and the first matching handler that responds wins. A dynamic route like `/users/:id` will also match `/users/me`, so if it is registered first it shadows the literal route. You register specific paths before generic ones.

Q: "What is `express.Router()` and why use it?"

A: A Router is a standalone, mountable middleware/route group — a mini Express app. You attach routes and router-level middleware to it and mount it under a base path with `app.use('/api/users', router)`. It keeps large apps modular: one router per resource, each in its own file.

Q: "A user hits `GET /users/me` but your handler returns the wrong record. Why?"

A: Almost certainly a route-ordering bug — `/users/:id` was registered before `/users/me`, so `:id` captured the literal string `"me"`. Move the specific route above the dynamic one.

Q (trap): "`req.params.id` for `/users/42` — is it the number 42?"

A: No, it is the string `"42"`. Route params (and query values) are always strings; comparing with `=== 42` fails. Convert and validate explicitly.

Q: "How would you attach authentication to only the admin routes?"

A: Put those routes in their own Router and register the auth middleware on that router with `router.use(authMiddleware)` (or as the second argument on the mount). It applies to the whole subtree without touching unrelated routes.

Cheat-card

- Route = **method + path + handler(s)**: `app.get('/users/:id', fn)`.
- `req.params` = path segments (`:id`); `req.query` = after `?`. Both are **strings**.
- Matching is **top-to-bottom, first responder wins** → specific before generic.
- `express.Router()` = mountable mini-app; `app.use('/base', router)` joins paths.
- Router paths are **relative** to the mount base.
- `app.route('/path')` chains `.get().put().delete()` on one path.
- `mergeParams: true` to read parent params in a nested router.
- Express 5 changed the path matcher — audit wildcard/regex paths on upgrade.

QUICK CHECK — no marks, just for you.

1. What is the difference between req.params and req.query?

- ○ req.params holds the query string; req.query holds path segments
- ○ req.params is for GET requests; req.query is for POST requests
- ○ req.params holds named segments from the path pattern (/users/:id); req.query holds the parsed query string after the ?
- ○ req.params is typed as numbers; req.query is typed as strings

Check answer

2. A user hits GET /users/me but gets the wrong record. What is the most likely bug?

- ○ The :id parameter was coerced to a number, dropping the "me" value
- ○ express.json() was missing, so the route could not read the path
- ○ The Router was mounted at the wrong base path
- ○ Route ordering: /users/:id was registered before /users/me, so :id captured the literal string "me"

3. For the route /users/:id and request /users/42, what is the type of req.params.id?

- ○ The string "42" — route params are always strings, so === 42 is false and you must convert with Number()
- ○ The number 42, parsed automatically by Express
- ○ A number if the segment is all digits, otherwise a string
- ○ undefined, because :id must be declared in req.query first

Check answer

‹ Previous
Express basics: what it is and why a framework

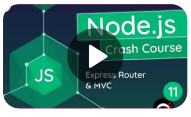
Next ›
Middleware: the pipeline, next(), and order

LEARN MORE

ARTICLES & DOCS

- Routing
  expressjs.com
- Basic routing
  expressjs.com

VIDEOS



Node.js Crash Course Tutorial #11 - Express Router & MVC
Net Ninja

Hand-picked references for this lesson.

Node.js Backend Kit